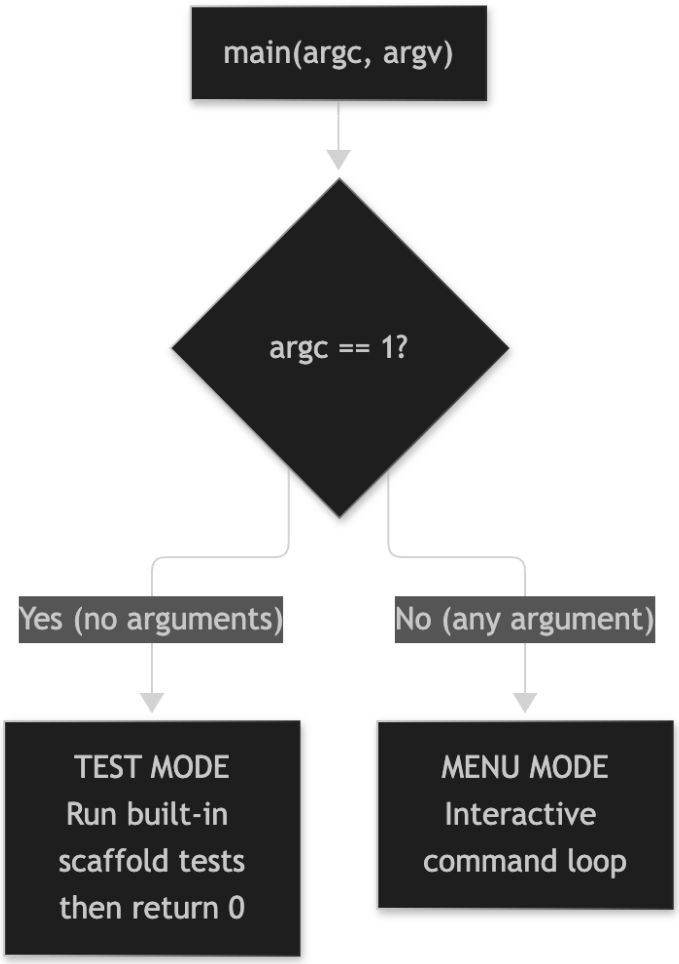
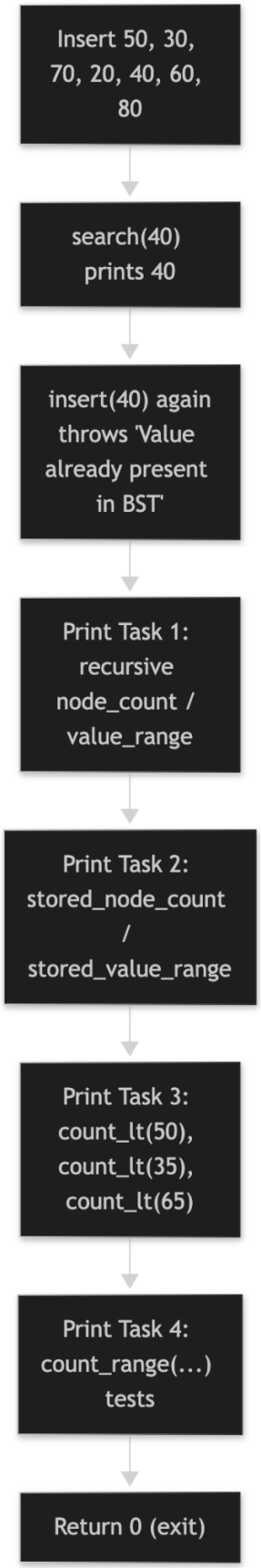
1. Top-Level Program Flow

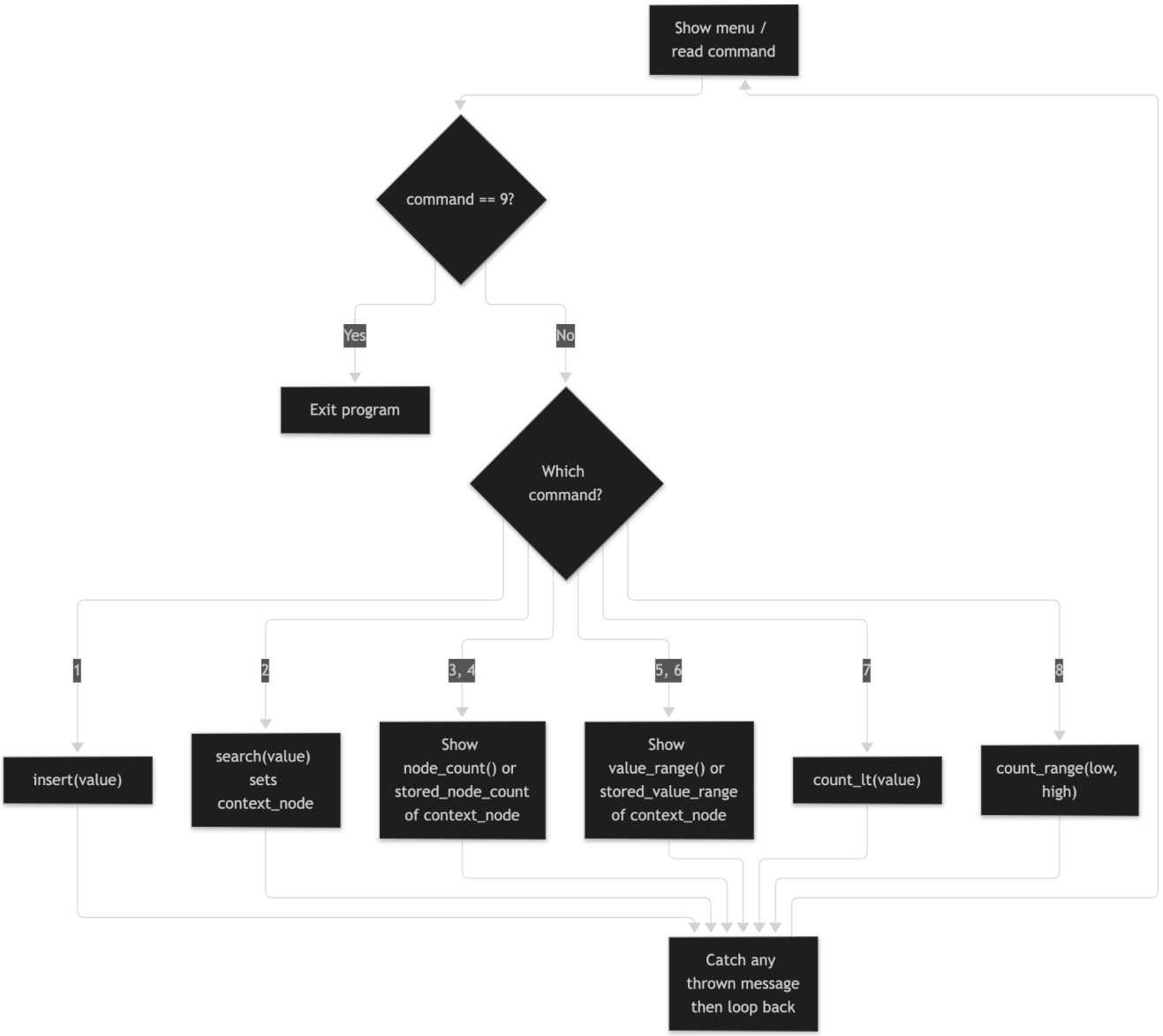
The entry point splits into two completely different worlds depending on how you launch the program.



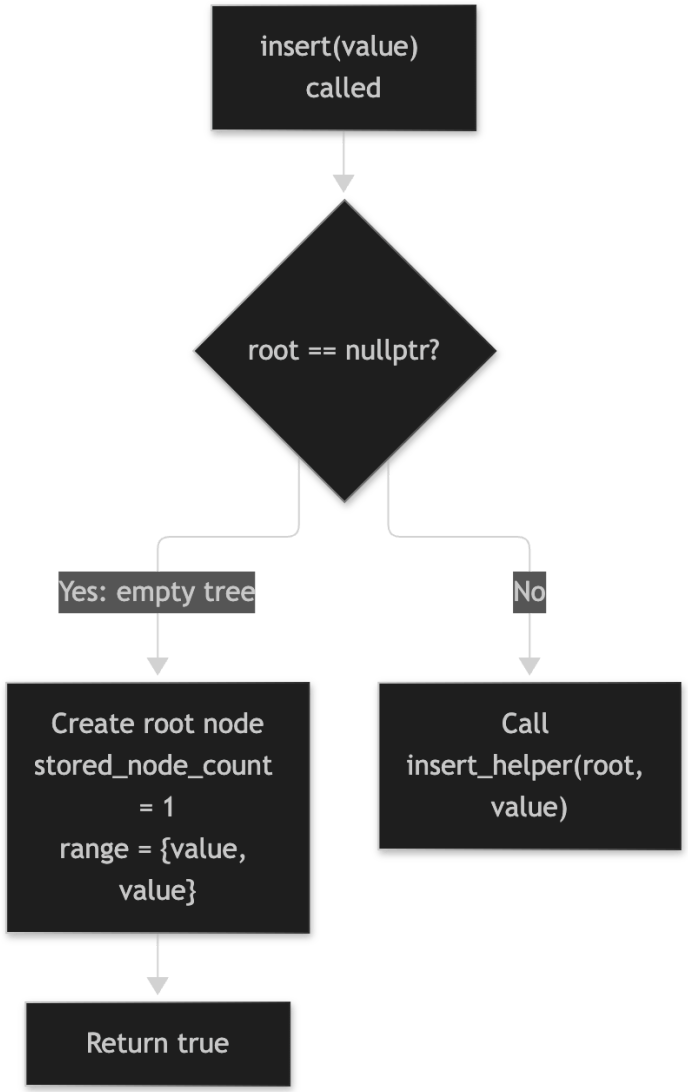
2. Test Mode (runs once, then exits)


Insert 50, 30, 70, 20, 40, 60, 80
search(40) prints 40
insert(40) again throws 'Value already present in BST'
Print Task 1: recursive node_count / value_range
Print Task 2: stored_node_count / stored_value_range
Print Task 3: count_lt(50), count_lt(35), count_lt(65)
Print Task 4: count_range(...) tests
Return 0 (exit)

3. Menu Mode (the interactive loop)

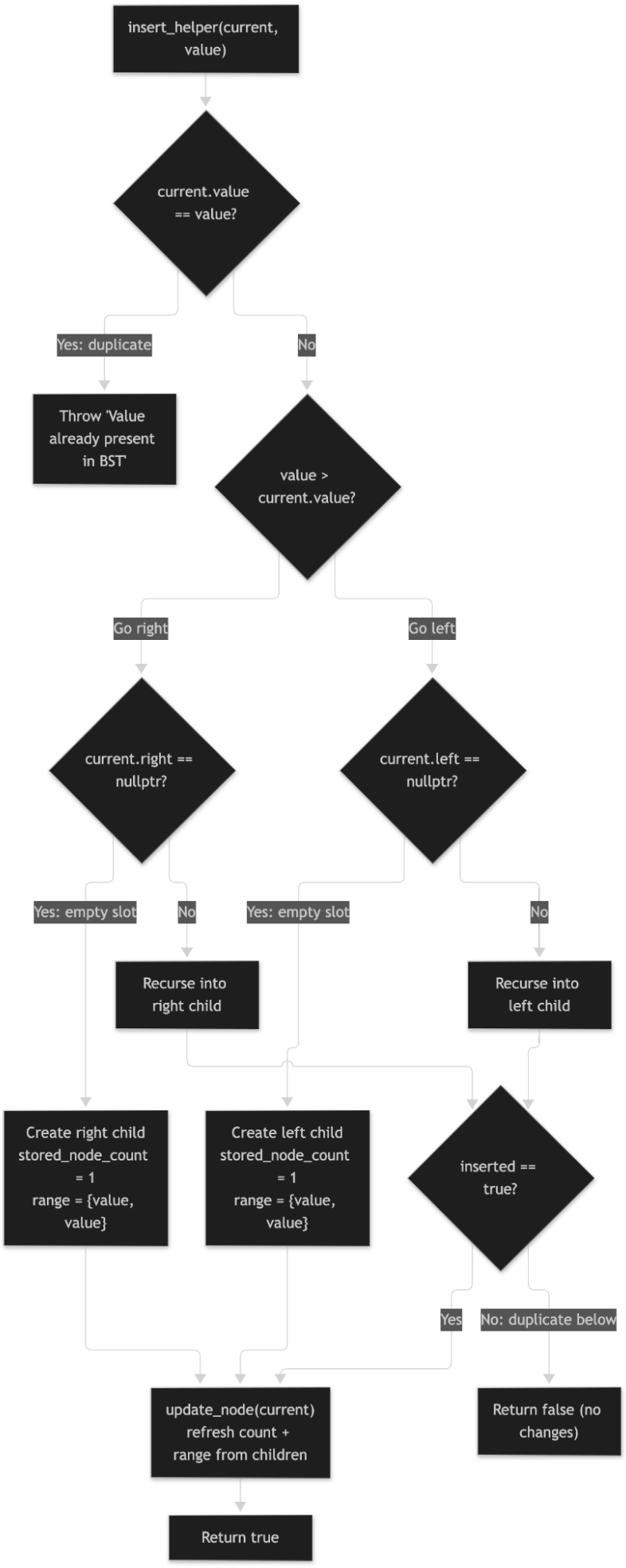


4. insert (value): The Two Entry Cases

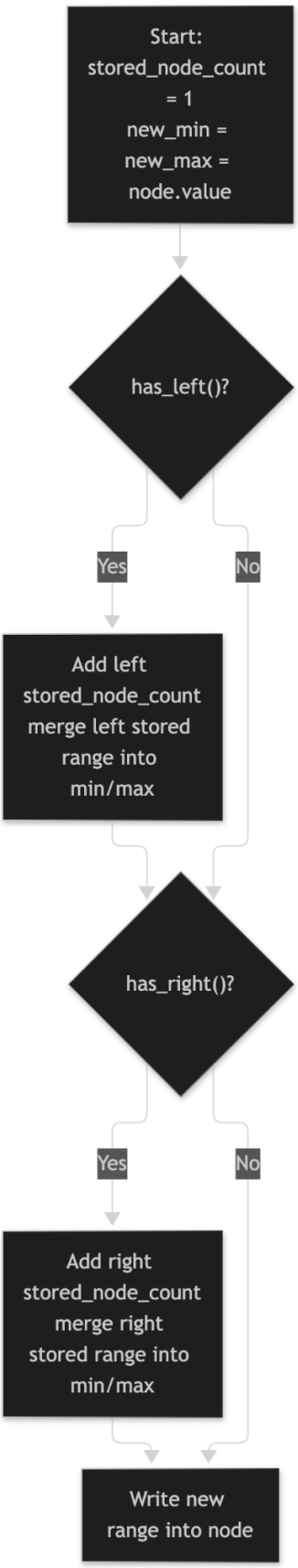


5. insert_helper: Descend, Create, Update on the Way Up

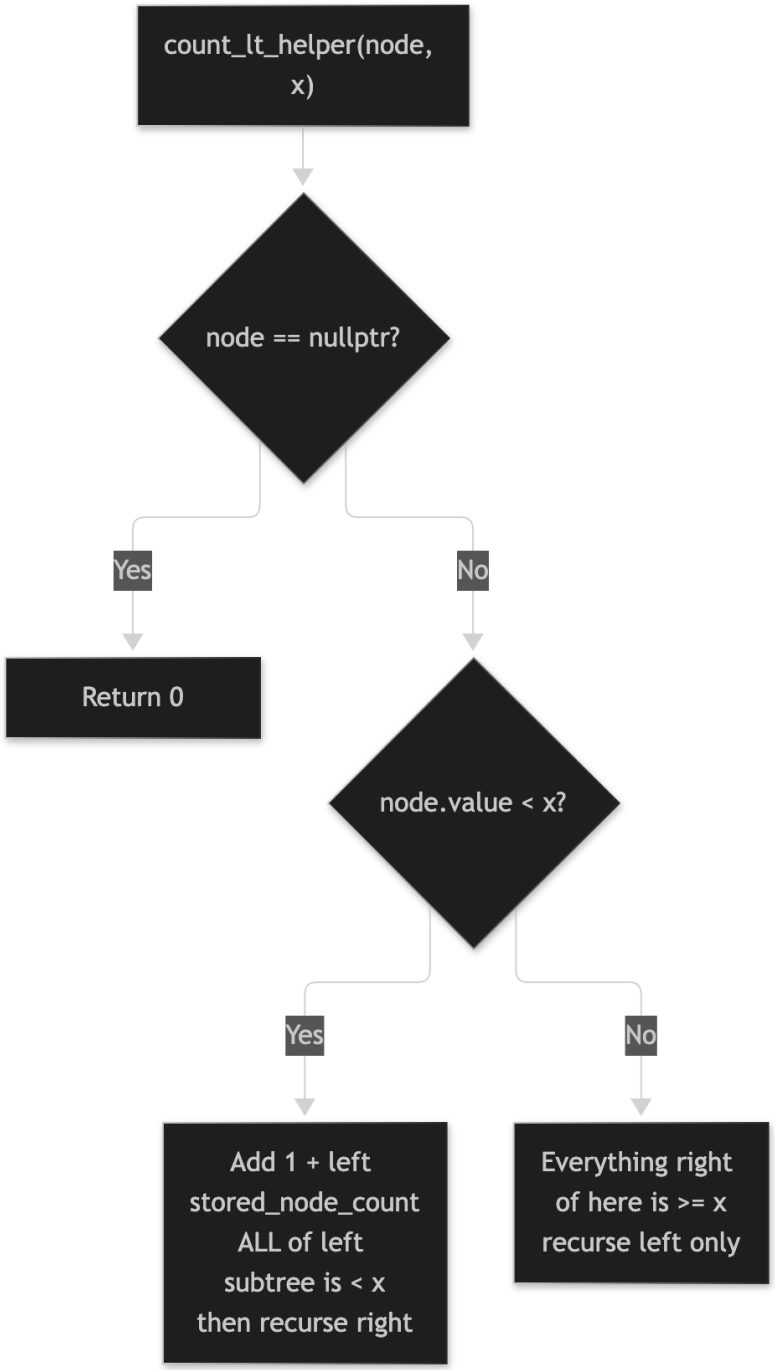
This is the heart of Task 2. Read it top to bottom for the descent, and notice update_node fires as the recursion unwinds.


insert_helper(current, value)
current.value == value?
Yes: duplicate
No
Throw 'Value already present in BST'
value > current.value?
Go right
Go left
current.right == nullptr?
current.left == nullptr?
Yes: empty slot
No
Yes: empty slot
No
Recurse into right child
Recurse into left child
Create right child stored_node_count = 1 range = {value, value}
Create left child stored_node_count = 1 range = {value, value}
inserted == true?
Yes
No: duplicate below
update_node(current) refresh count + range from children
Return false (no changes)
Return true

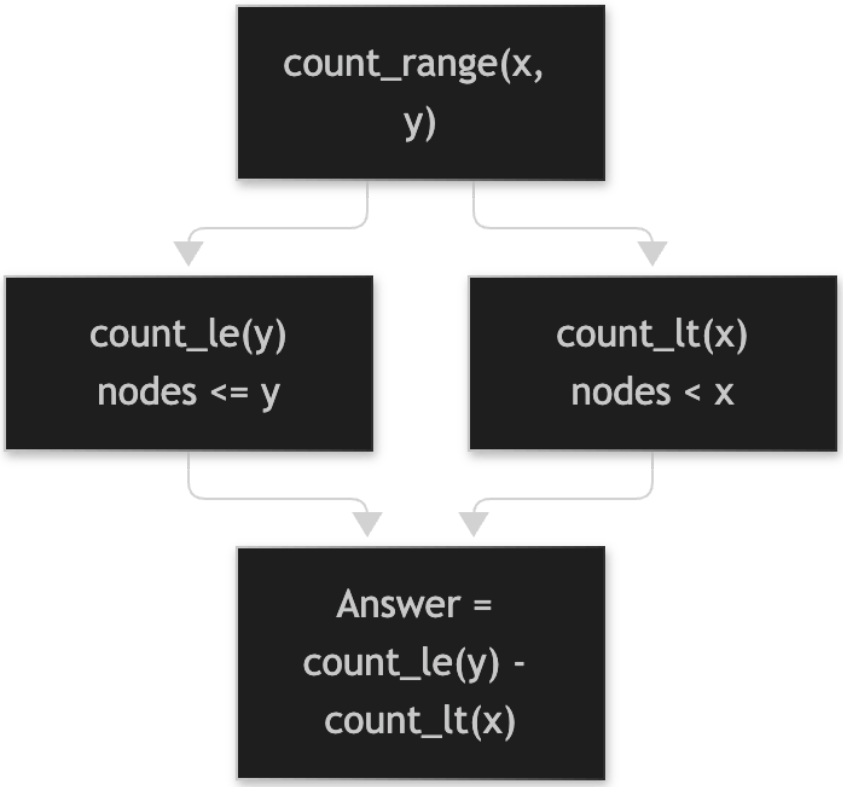
6. update_node (node): Recompute Aggregates From Children



7. count_lt (x): Skip Whole Subtrees in O(h)



8. count_range (x, y): 2 Cheap Counts Combined



Reading Guide

1. Diagrams 1-3 cover program structure: launch mode, test sequence, and the menu loop.
2. Diagrams 4-6 are the Task 2 story: descend to a leaf, create the node, then repair aggregates on every ancestor while unwinding, which costs O(h).
3. Diagrams 7-8 are the Tasks 3–4 story: never count node-by-node; use stored_node_count to absorb entire subtrees in one step, keeping every query at O(h).